

SQL FUNDAMENTALS: SELECT & FILTERING ① Selecting and Sorting

employees

id	first_name	last_name	department	salary	hire_date	city
1	Alice	Green	IT	70000	2020-01-10	Johannesburg
2	Brian	Lee	HR	45000	2019-03-22	Cape Town
3	Cathy	Zulu	Finance	65000	2018-07-18	Durban
4	David	Mokoena	Marketing	50000	2021-11-05	Pretoria
5	Eva	Naidoo	IT	72000	2017-09-30	Johannesburg

01. Retrieve all columns from the employees table.

SELECT *

FROM employees;

id	first_name	last_name	department	salary	hire_date	City
1	Alice	Green	IT	70000	2020-01-10	Johannesburg
2	Brian	Lee	HR	45000	2019-03-22	Cape Town
3	Cathy	Zulu	Finance	65000	2018-07-18	Durban
4	David	Mokoena	Marketing	50000	2021-11-05	Pretoria
5	Eva	Naidoo	IT	72000	2017-09-30	Johannesburg

02. Find all unique departments.

SELECT DISTINCT (department)

FROM employees;

department
IT
HR
Finance
Marketing

03. Retrieve first and last names ordered by salary descending.

```
SELECT first_name,  
       last_name,  
       salary  
FROM employees  
ORDER BY salary DESC;
```

first_name	last_name	salary
Eva	Naidoo	72000
Alice	Green	70000
Cathy	Zulu	65000
David	Mokoena	50000
Brian	Lee	45000

04. Retrieve the top 3 highest-paid employees.

```
SELECT id,  
       first_name,  
       last_name,  
       salary  
FROM employees  
ORDER BY salary DESC LIMIT 3;
```

id	first_name	last_name	salary
5	Eva	Naidoo	72000
1	Alice	Green	70000
3	Cathy	Zulu	65000

② Filtering with ~~where~~ WHERE

05. Find employees in the IT department.

```
SELECT id,  
       first_name,  
       last_name,  
       department
```

```
FROM employees
```

```
WHERE department = 'IT';
```

id	first_name	last_name	department
1	Alice	Green	IT
5	Eva	Naidoo	IT

06. Find employees in Finance with salary > 6000

```
SELECT id,  
       first_name,  
       last_name,  
       department,  
       salary
```

```
FROM employees
```

```
WHERE department = 'Finance' AND salary > 60000;
```

id	first_name	last_name	department	salary
1	Alice	Green	IT	70000
3	Cathy	Zulu	Finance	65000
5	Eva	Naidoo	IT	70000

07. Find employees in HR or Marketing.

```
SELECT id,
       first_name,
       last_name,
       department
FROM employees
WHERE department IN ('HR', 'Marketing');
```

id	first_name	last_name	department
2	Brian	Lee	HR
4	David	Mokcena	Marketing

08. Find employees not in IT.

```
SELECT id,
       first_name,
       last_name,
       department
FROM employees
WHERE department NOT 'IT';
```

id	first_name	last_name	department
2	Brian	Lee	HR
3	Cooby	Zulu	Finance
4	David	Mokcena	Marketing

09. Find employees in IT, HR or Financing using IN.

```
SELECT id,  
       first_name,  
       last_name,  
       department
```

```
FROM employees
```

```
WHERE department IN ('IT', 'HR', 'Finance');
```

id	first_name	last_name	department
1	Alice	Green	IT
2	Brian	Lee	HR
3	Cathy	Zulu	Finance
5	Eva	Naidoo	IT

10. Find employees in IT with Salary > 65000 and city Johannesburg.

```
SELECT id,  
       first_name,  
       last_name,  
       department,  
       salary,  
       city
```

```
FROM employees
```

```
WHERE department = 'IT' AND salary > 65000 AND city = 'Johannesburg';
```

id	first_name	last_name	department	salary	city
1	Alice	Green	IT	70000	Johannesburg
5	Eva	Naidoo	IT	72000	Johannesburg

SQL Aggregate Functions & Operators

01. Students

Student-id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

01. List all distinct departments in the Student table.

```
SELECT DISTINCT (department)
```

```
FROM students;
```

department
IT
HR
Finance

02. Get the average age of students per department.

```
SELECT AVG (age) AS avg-age,  
       department
```

```
FROM students
```

```
GROUP BY department;
```

department	avg-age
IT	20.5
HR	22
Finance	23

Q3. Show departments with more than 1 student.

```
SELECT COUNT(student_id) AS student_count,  
       department
```

```
FROM students
```

```
GROUP BY department
```

```
HAVING COUNT(student_id) > 1;
```

department	student_count
IT	2
HR	2

Q4. Get all students whose age is between 21 and 23.

```
SELECT student_id,
```

```
       name,
```

```
       age,
```

```
       department
```

```
FROM students
```

```
WHERE age BETWEEN 21 AND 23;
```

student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

Q5. List all students in the IT or HR department who are older than 21.

```
SELECT student_id,
       name,
       age,
       department
```

```
FROM students
```

```
WHERE department IN ('IT', 'HR') AND age > 21;
```

student_id	name	age	department
2	Bob	22	HR
4	Quinn	23	HR
5	Eve	22	HR

Q2. COURSES

course_id	course_name	department	credits
101	SQL Basics	IT	3
102	Python	IT	4
103	Data Science	IT	4
104	Excel	Finance	2
105	Statistics	HR	3

Q6. Show total credits per department, only for departments with more than 5 total credits.

```
SELECT department,
```

```
       SUM(credits) AS total_credits
```

```
FROM courses
```

```
GROUP BY department
```

```
HAVING SUM(credits) > 5;
```

department	total_credits
IT	11

Q7. List all courses that do not have 4 credits.

```
SELECT course_id,  
       course_name,  
       department,  
       credits
```

```
FROM courses
```

```
WHERE credits < 4;
```

course_id	course_name	department	credits
101	SQL Basic	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

Q8. Show the top 3 courses by credits in descending order.

```
SELECT course_id,  
       course_name,  
       credits
```

```
FROM courses
```

```
ORDER BY credits DESC LIMIT 3;
```

course_id	course_name	credits
102	Python	4
103	Data Science	4
101	SQL Basic	3

03. enrollments

enrollment_id	student_id	course_id	grade
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

09. Get the maximum, minimum, and average grade across all enrollments

SELECT MAX(grade) AS max_grade

FROM enrollments;

max_grade
90

SELECT MIN(grade) AS min_grade

FROM enrollments;

min_grade
78

SELECT AVG(grade) AS avg_grade

FROM enrollments;

avg_grade
49.4

10. Count how many enrollments exist per course.

```
SELECT COUNT (enrollment-id) AS enrollment-count,  
       course-id  
FROM enrollments  
GROUP BY course-id;
```

enrollment-count	course-id
1	101
2	102
3	103
4	104
5	105

04. Salaries

employee-id	name	department	salary	bonus
1	Tom	IT	60000	5000
2	Jerry	HR	55000	4000
3	Spike	Finance	70000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

11. Find total salary and total bonus per department.

```
SELECT department,  
       SUM(salary) AS total-salary,  
       SUM(bonus) AS total-bonus  
FROM salaries  
GROUP BY department;
```

department	total - salary	total - bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

12. Show departments where average salary is above 55,000,

```
SELECT department,
       AVG(salary) AS avg-salary
```

```
FROM salaries
```

```
GROUP BY department
```

```
HAVING AVG(salary) > 55,000;
```

department	avg-salary
IT	61,000
Finance	70,000

13. List employees whose salary plus bonus is greater than 60,000.

```
SELECT employee-id,
       name,
       salary,
       bonus,
       (salary + bonus) AS total-compensation
```

```
FROM salaries
```

```
WHERE (salary + bonus) > 60,000;
```

employee-id	name	salary	bonus	total-compensation
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

05. Projects

project-id	project-name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
3	Dashboard	IT	150000
4	Website	Marketing	60000
5	HR Portal	HR	50000

14. Show total and average budget per department. Only include departments with average budget above 70,000.

```

SELECT department,
       SUM(budget) AS total-budget,
       AVG(budget) AS avg-budget
FROM projects
GROUP BY department
HAVING AVG(budget) > 70,000;

```

department	total-budget	avg-budget
IT	270,000	135,000
Finance	80,000	80,000

15. List all projects with budgets between 50,000 and 120,000, excluding the Marketing department.

SELECT *

FROM projects

WHERE budget BETWEEN 50,000 AND 120,000 AND department NOT 'Marketing';

project_id	project_name	department	budget
1	AI App	IT	120 000
2	Payroll System	Finance	80 000
3	Dashboard	IT	150 000